# Lab3-Score Distillation Sampling Implementation

Chen En Lo (114550195)
NYCU
asciibase64.cs14@nycu.edu.tw

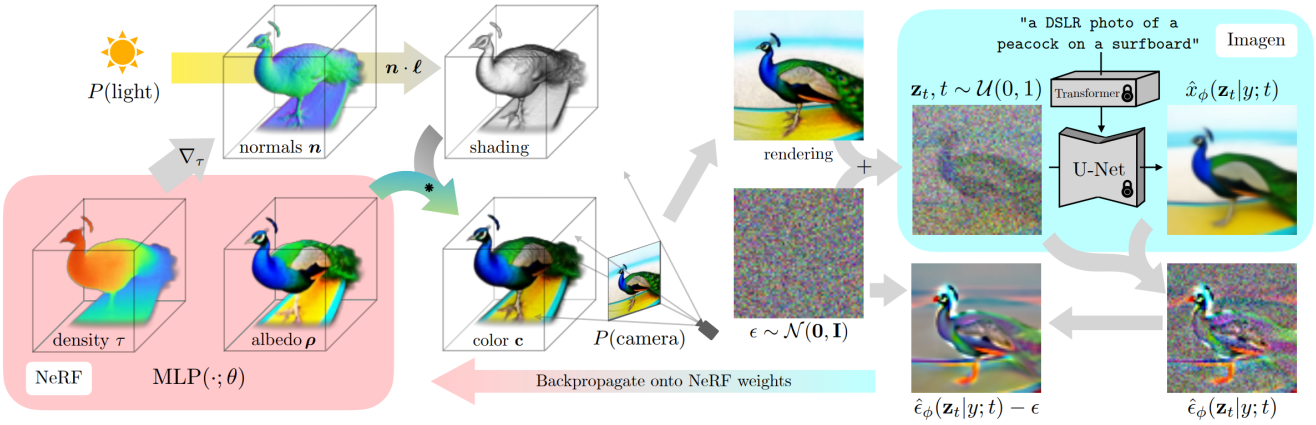


Figure 1. **Overview of score distillation.** A pretrained diffusion model predicts the noise added to a rendered image, and the resulting score difference guides the optimization of NeRF parameters. Figure from Poole et al. [2].

## 1. Introduction

Diffusion models generate images by progressively denoising random noise. In contrast, score distillation uses a pretrained diffusion model to provide gradients that guide the optimization of a separate representation, rather than directly sampling from the diffusion model. This enables diffusion models to guide the generation of content represented by other parameterizations.

As illustrated in Fig. 1, score distillation uses a pretrained diffusion model to guide the optimization of a parameterized representation. The representation is converted into an image, and the diffusion model provides guidance based on a text prompt to iteratively update its parameters.

Score distillation has been widely used in 3D generation tasks. In this lab, however, we apply score distillation to 2D image generation rather than 3D object generation. Specifically, we implement and compare three variants: Score Distillation Sampling (SDS), Variational Score Distillation (VSD), and Score Distillation via Inversion (SDI).

We evaluate the generated images quantitatively using CLIP scores and qualitatively through visual comparisons. Additionally, we conduct ablation studies on the guidance scale in SDS, as well as the number of inversion steps and target update interval in SDI, to investigate how these hyperparameters affect generation quality.

## 2. Methodology

Given a text prompt $y$, we generate an image by optimizing a latent $\mathbf{x}_0 \in \mathbb{R}^{1\times4\times64\times64}$ initialized from $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Stable Diffusion 2.1-base provides the pretrained noise predictor $\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t)$. Its base parameters remain fixed, and the final latent is decoded into an RGB image using the pretrained VAE.

Let $t$ denote a diffusion timestep and $\bar{\alpha}_t$ the cumulative product of the noise-schedule coefficients. Gaussian forward noising produces

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{1}$$

We obtain the coefficients from the model's DDIM scheduler, with timestep bounds $t_{\min} = 20$ and $t_{\max} = 980$ out of 1000 training timesteps. The sampled noise $\boldsymbol{\epsilon}_t$ is distinguished from the predicted noise $\hat{\boldsymbol{\epsilon}}_\theta$.

The implementation applies classifier-free guidance (CFG) by combining predictions conditioned on the positive prompt $y$ and negative prompt $y_-$, with guidance weight $\omega$:

$$\hat{\boldsymbol{\epsilon}}_{\mathrm{CFG}}(\mathbf{x}_t, y, t) = \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y_-, t) + \omega\big[\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t) - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y_-, t)\big]. \tag{2}$$

Both text embeddings are computed without gradients, and the two predictions are evaluated in a single batched U-Net call. Below, $\hat{\boldsymbol{\epsilon}}_\theta$ denotes the prediction after CFG with the guidance setting specified for each method.

### 2.1. Score Distillation Sampling (SDS)

**Concept.** SDS [2] uses the difference between predicted and sampled noise to update a parameterized representation. Let $\boldsymbol{\theta}$ denote the optimized representation parameters, $c$ a camera viewpoint, and $\mathbf{x}_0 = g(\boldsymbol{\theta}; c)$ the rendered image. Omitting the noise predictor Jacobian gives

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{SDS}}(\boldsymbol{\theta}) = \big(\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t) - \boldsymbol{\epsilon}_t\big) \frac{\partial \mathbf{x}_t}{\partial \boldsymbol{\theta}}. \tag{3}$$

The subscript $\theta$ on the noise predictor identifies the pretrained network, while $\boldsymbol{\theta}$ in the gradient denotes the parameters being optimized.

**Implementation.** At each iteration, we sample an integer $t$ uniformly from $[20, 980)$, draw Gaussian noise, and construct $\mathbf{x}_t$ using Eq. (1). Our implementation additionally weights the noise residual by $w(t) = 1 - \bar{\alpha}_t$ and applies the weighted residual directly as the latent gradient. The noise prediction runs under `torch.no_grad()`. We call `detach()` on the weighted residual and return its elementwise product with the latent, summed to a scalar. Backpropagation through this scalar supplies the chosen gradient without differentiating through the U-Net.

```
def get_sds_loss(self, latents, text_embeddings, guidance_scale
    =7.5):
   t = torch.randint(self.min_step, self.max_step, (latents.
       shape[0],), device=latents.device)
   noise = torch.randn_like(latents)
   alpha_bar_t = self.alphas[t].view(-1, 1, 1, 1)
   xt = torch.sqrt(alpha_bar_t) * latents + torch.sqrt(1 -
       alpha_bar_t) * noise
   with torch.no_grad():
      noise_pred = self.get_noise_preds(xt, t, text_embeddings,
          guidance_scale)
   w = 1 - alpha_bar_t
   grad = (w * (noise_pred - noise)).detach()
   loss = (grad * latents).sum()
   return loss
```

### 2.2. Variational Score Distillation (VSD)

**Concept.** VSD [4] learns a noise predictor for the current generated samples to address over-smoothing and over-saturation. The sampled-noise term in SDS is replaced by the LoRA-adapted prediction:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{VSD}}(\boldsymbol{\theta}) = \big(\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t) - \hat{\boldsymbol{\epsilon}}_{\mathrm{LoRA}}(\mathbf{x}_t, y, t)\big) \frac{\partial \mathbf{x}_t}{\partial \boldsymbol{\theta}}. \tag{4}$$

We optimize one latent per prompt. LoRA is trained with a denoising mean squared error between its predicted noise and the Gaussian noise used to construct $\mathbf{x}_t$.

**Implementation.** We disable the adapters for the frozen teacher prediction, then enable them for the trainable prediction on the same noisy latent and timestep. The adapted predictor uses $\omega = 1$ so that CFG reduces to the positive-condition prediction, allowing LoRA to learn the conditional denoising prediction without guidance amplification.

As in SDS, the latent update uses the noise residual weighted by $1 - \bar{\alpha}_t$. The residual is detached before forming the scalar used for latent optimization. The adapter's denoising loss is added with `lora_loss_weight=1.0`, and its input latent is detached. These operations ensure that the distillation term updates the latent and the denoising term updates only LoRA.

```
def get_vsd_loss(self, latents, text_embeddings, guidance_scale
    =7.5, lora_loss_weight=1.0):
   noise = torch.randn_like(latents)
   t = torch.randint(self.min_step, self.max_step, (latents.
       shape[0],), device=latents.device)
   alpha_bar_t = self.alphas[t].view(-1, 1, 1, 1)
   xt = torch.sqrt(alpha_bar_t) * latents + torch.sqrt(1 -
       alpha_bar_t) * noise
   with torch.no_grad():
      self.unet.disable_adapters()
      noise_pred = self.get_noise_preds(xt, t, text_embeddings,
          guidance_scale)
      self.unet.enable_adapters()
   noise_lora = self.get_noise_preds(xt.detach(), t,
       text_embeddings, 1.0)
   loss_lora = F.mse_loss(noise_lora, noise)
   w = 1 - alpha_bar_t
   grad = (w * (noise_pred - noise_lora)).detach()
   loss = (grad * latents).sum() + loss_lora * lora_loss_weight
   return loss
```

### 2.3. Score Distillation via Inversion (SDI)

**Concept.** SDI [1] obtains a noisy sample by DDIM inversion [3] instead of injecting independently sampled Gaussian noise. The current latent is updated by minimizing the mean squared error to a denoised target:

$$\mathcal{L}_{\mathrm{SDI}} = \mathrm{mean}\big[(\mathbf{x}_0 - \mathbf{x}_{0|t})^2\big]. \tag{5}$$

Here, $\mathbf{x}_{0|t}$ is the cached clean estimate reconstructed directly from the inverted $\mathbf{x}_t$ at the same timestep $t$. The square is elementwise, and the mean averages over all latent elements. The timestep follows a rounded

linear schedule from 980 to 20 over the optimization iterations rather than being sampled independently.

**DDIM inversion.** We implement the inversion loop explicitly without calling a library inversion utility. The loop starts from the current clean latent and traverses rounded, evenly spaced timesteps from zero to the target timestep. At each step, it predicts noise, estimates the clean latent, and combines that estimate with the predicted noise using the coefficients of the next scheduled timestep. The initial coefficient is set to 1 because the input is clean. Inversion is deterministic: the accepted `eta` argument is unused, as permitted by the assignment.

```
@torch.no_grad()
def invert_noise(self, latents, target_t, text_embeddings,
     guidance_scale=-7.5, n_steps=10, eta=0.3):
    if n_steps < 1:
        raise ValueError("n_steps must be at least 1")
    target_t = torch.as_tensor(target_t, device=latents.device).
         expand(latents.shape[0])
    stepsize = target_t / n_steps
    timesteps = (
        torch.arange(n_steps + 1, device=latents.device)[:, None]
             * stepsize
    ).round().long()
    alphas = self.alphas[timesteps].to(latents.dtype).view(
        n_steps + 1, latents.shape[0], 1, 1, 1
    )
    alphas[0] = 1
    sqrt_alphas = alphas.sqrt()
    sqrt_one_minus_alphas = (1 - alphas).sqrt()
    for i in range(n_steps):
        noise_pred = self.get_noise_preds(
            latents, timesteps[i], text_embeddings,
                guidance_scale
        )
        pred_x0 = (
            latents - sqrt_one_minus_alphas[i] * noise_pred
        ) / sqrt_alphas[i]
        latents = (
            sqrt_alphas[i + 1] * pred_x0
            + sqrt_one_minus_alphas[i + 1] * noise_pred
        )
    return latents
```

**SDI Implementation.** The implemented target is the clean estimate reconstructed directly from the inverted noisy latent:

$$\mathbf{x}_{0|t} = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, y, t)}{\sqrt{\bar{\alpha}_t}}. \tag{6}$$

```
def get_sdi_loss(
    self,
    latents,
    text_embeddings,
    guidance_scale=7.5,
    current_iter=0,
    total_iters=500,
    inversion_guidance_scale=-7.5,
    inversion_n_steps=10,
    inversion_eta=0.3,
    update_interval=25,
):
    B = latents.shape[0]
    progress = min(max(current_iter / max(total_iters - 1, 1),
         0.0), 1.0)
    timestep = round(self.max_step + (self.min_step - self.
         max_step) * progress)
    t = torch.full((B,), timestep, device=latents.device, dtype=
         torch.long)
    should_update = (current_iter % update_interval == 0) or not
         hasattr(self, 'sdi_target')
    if should_update:
        with torch.no_grad():
            latents_noisy = self.invert_noise(
                latents, t, text_embeddings,
                guidance_scale=inversion_guidance_scale,
                n_steps=inversion_n_steps,
                eta=inversion_eta
            )
            noise_pred = self.get_noise_preds(latents_noisy, t,
                 text_embeddings, guidance_scale)
            alpha_bar_t = self.alphas[t].to(latents.dtype).view
                 (-1, 1, 1, 1)
            target = (
                latents_noisy - torch.sqrt(1 - alpha_bar_t) *
                     noise_pred
            ) / torch.sqrt(alpha_bar_t)
            self.sdi_target = target.detach()
    loss = F.mse_loss(self.sdi_target, latents)
    return loss
```

Table 1. **CLIP score** (↑, ViT-B/32) per prompt.

| Prompt | SDS | VSD | SDI |
|---|---|---|---|
| A red bus driving on a desert road | 0.3442 | 0.3528 | **0.3846** |
| a boat in a river | 0.2953 | **0.3034** | 0.2992 |
| A cabin surrounded by forests | 0.3010 | 0.3228 | **0.3247** |
| A church beside a lake | 0.3246 | 0.3224 | **0.3362** |
| A villa close to the pool | 0.2935 | **0.3097** | 0.3006 |
| A castle next to a river | 0.2959 | **0.3009** | 0.3008 |
| A burger on the table | **0.3047** | 0.2874 | 0.2990 |
| A dog sitting on grass | **0.2880** | 0.2688 | 0.2627 |
| a cat sitting on a table | 0.2989 | 0.2725 | **0.3021** |
| A car on the road | **0.2953** | 0.2699 | 0.2757 |
| **Average** | 0.3042 | 0.3011 | **0.3086** |

## 3. Experiments & Results

### 3.1. Quantitative Results

Table 1 reports CLIP scores for the ten evaluation prompts. SDI has the highest average CLIP score (0.3086), followed by SDS (0.3042) and VSD (0.3011). SDI leads on four prompts, while SDS and VSD each lead on three.

The score ranking only partly agrees with the visual comparison in Fig. 2. For the burger, SDS scores 0.3047 despite producing a blurrier image than SDI, which scores 0.2990. The object remains recognizable in SDS, illustrating that stronger image–text similarity does not necessarily imply sharper details.

### 3.2. Qualitative Results

Figure 2 shows clear differences in texture and structure among the three methods. SDS produces recognizable objects and simple compositions, but edges and surfaces are heavily smoothed. For example, the burger has a clear silhouette but little texture, and the cabin and castle lose fine architectural details.

Figure 2. **Qualitative comparison** of SDS, VSD and SDI on the 10 evaluation prompts.

Figure 3. **Ablation on SDS `guidance_scale`.**

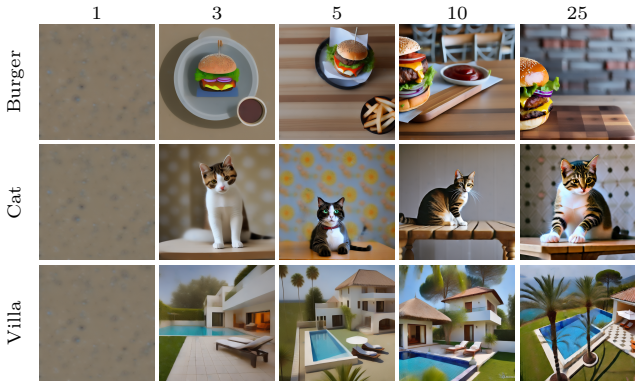


Figure 4. **Ablation on SDI `inversion_n_steps`.**

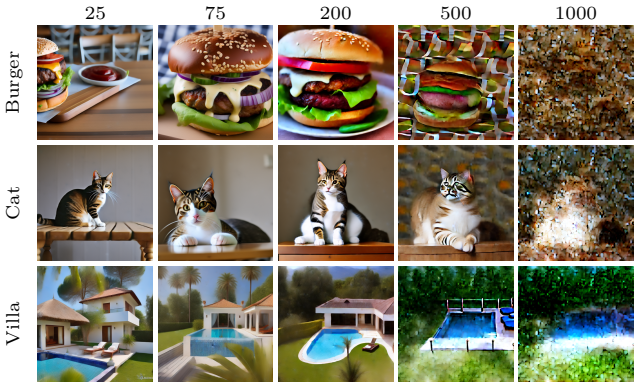


Figure 5. **Ablation on SDI `sdi_update_interval`.**

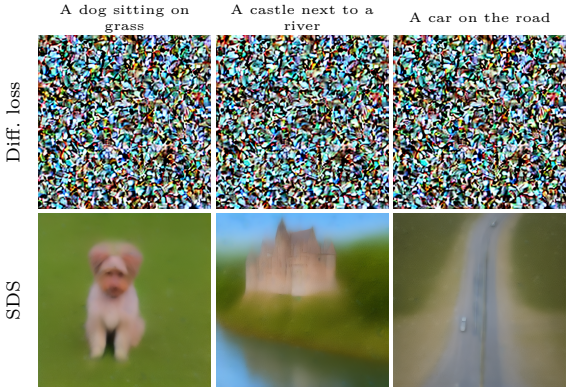


Figure 6. **Original diffusion loss** (top) vs. SDS (bottom).

Table 2. **Ablation study.** Average CLIP score (↑) over the Burger, Cat, and Villa prompts. Existing baseline images are reused.

| *(a) SDS: `guidance_scale`* | | | | | |
|---|---|---|---|---|---|
| Value | 7.5 | 15 | 25 | 50 | 100 |
| CLIP | 0.2426 | 0.2950 | **0.2991** | 0.2879 | 0.2815 |
| *(b) SDI: `inversion_n_steps`* | | | | | |
| Value | 1 | 3 | 5 | 10 | 25 |
| CLIP | 0.1809 | 0.2942 | 0.2874 | **0.3006** | 0.2956 |
| *(c) SDI: `sdi_update_interval`* | | | | | |
| Value | 25 | 75 | 200 | 500 | 1000 |
| CLIP | **0.3006** | 0.2848 | 0.2912 | 0.2739 | 0.1934 |

VSD produces sharper local details and more varied colors, but many images contain dense, irregular textures. The burger has visible sesame seeds and layered ingredients, while the villa includes vegetation and pool reflections. However, mottled patterns cover the burger background, and the villa's walls and roof appear fragmented. These artifacts show that additional detail does not always yield coherent surfaces.

SDI generally produces cleaner surfaces and more distinct object boundaries. The cabin and church have clearer architectural structure, and the burger shows individual ingredients against a smoother background.

### 3.3. Ablation Study

We varied one hyperparameter at a time while keeping the remaining settings in Tab. 3 fixed. Each entry in Tab. 2 averages CLIP scores over the Burger, Cat, and Villa prompts.

#### 3.3.1. SDS: `guidance_scale`

Table 2(a) shows that the score rises from 0.2426 at guidance 7.5 to 0.2950 at 15, with the highest observed score of 0.2991 at 25. Larger scales reduce the score to 0.2879 at 50 and 0.2815 at 100. In Fig. 3, guidance 7.5 produces blurred subjects, and the villa is barely visible around the pool. At 15, the subjects become more distinct, while 50 and 100 introduce increasingly dense background textures, especially around the burger and cat.

Increasing CFG amplifies the difference between positive- and negative-condition noise predictions, giving the noisy SDS updates a stronger direction toward the text prompt. Excessively strong guidance can also exaggerate textures, consistent with the cluttered backgrounds at 50 and 100. Guidance 25 gives the best observed score among the tested values, compared with 7.5 commonly used for ordinary sampling. These results suggest that SDS benefits from stronger text guidance, but increasing it beyond a moderate value does not necessarily improve alignment or visual quality.

#### 3.3.2. SDI: `inversion_n_steps`

One inversion step produces a low score of 0.1809 and largely unrecognizable images in Fig. 4, consistent with an overly coarse inversion. Three steps recover recognizable subjects and raise the score to 0.2942. The scores then vary non-monotonically: 0.2874 at 5 steps, 0.3006 at 10, and 0.2956 at 25. More steps therefore do not guarantee better text alignment. Smaller inversion steps provide a finer approximation to the predicted trajectory, but final image quality also depends on target reconstruction and latent optimization.

Each additional inversion step requires another U-Net evaluation whenever the target is refreshed, so runtime grows approximately linearly with the step count. The trade-off is clear: 1 step is fast but fails to form recognizable subjects, whereas 25 steps costs about 6.7 times as much as 3 steps for a score increase of only about 0.0013. Three steps is therefore a lower-cost option with a similar CLIP score, while 10 steps gives the highest observed score among the tested settings.

#### 3.3.3. SDI: `sdi_update_interval`

The highest observed score is 0.3006 at interval 25, while interval 1000 drops to 0.1934 (Tab. 2(c)).

At the smallest tested interval of 25, the target is refreshed 40 times during 1000 iterations, allowing it to follow the changing latent and incorporate progressively lower noise levels. The subjects remain recognizable in Fig. 5, and this setting gives the highest observed score. At the largest interval of 1000, the target is constructed only once, at the initial timestep 980. All subsequent iterations fit that fixed target without receiving a new estimate at a lower noise level, consistent with the severe artifacts and low score. Larger intervals therefore hold potentially outdated targets for more iterations and skip more scheduled timesteps. The results favor repeated target updates over fitting a single initial target, although the intermediate scores show that the effect is not strictly monotonic.

### 3.4. Hyperparameters

Table 3 lists the settings used for the main comparison. The ablations vary only the parameter under study; all other settings remain fixed.

## 4. Discussion

### 4.1. Why the Original Diffusion Loss Fails

For the three prompts in Fig. 6, we directly minimized the noise-prediction MSE by backpropagating through the frozen U-Net, retaining its input Jacobian.

Noise-prediction error measures how well the model recovers a known perturbation; it does not directly

Table 3. **Hyperparameters** for each task.

| Hyperparameter | SDS | VSD | SDI |
|---|---|---|---|
| `guidance_scale` | 25 | 7.5 | 7.5 |
| `lr` | 0.01 | 0.01 | 0.005 |
| `steps` | 500 | 500 | 1000 |
| `lora_lr` | – | 1e-4 | – |
| `lora_rank` | – | 4 | – |
| `lora_loss_weight` | – | 1.0 | – |
| `inversion_n_steps` | – | – | 10 |
| `inversion_guidance_scale` | – | – | -7.5 |
| `inversion_eta` | – | – | 0.3 |
| `sdi_update_interval` | – | – | 25 |

measure whether the clean latent represents a realistic image. When the latent is optimized, the U-Net Jacobian transforms the noise residual into an update. This transformation can amplify sensitivity to local patterns rather than produce a coherent change toward the requested object. SDS omits that term and uses the noise residual directly as guidance.

In Fig. 6, the diffusion-loss results contain dense, colorful noise without a recognizable dog, castle, or car, whereas SDS produces recognizable, though blurred, subjects. The average CLIP score over these three prompts is 0.1831 for diffusion loss versus 0.2931 for SDS. The images are consistent with optimizing denoising error without obtaining coherent object structure.

### 4.2. Why SDI and VSD Outperform SDS

**SDS.** In Fig. 2, SDS produces recognizable but heavily smoothed subjects, such as the burger and cabin, with little surface texture or fine structural detail. SDS samples fresh Gaussian noise and a timestep at each iteration, so successive updates can favor different image changes. These fluctuating directions can wash out fine details during optimization. Strong CFG reinforces the prompt, but does not ensure that these details are preserved.

**VSD.** VSD still adds Gaussian noise, but replaces that sampled noise in the distillation residual with the prediction of a LoRA model trained to denoise the current generated latent. The update therefore compares the pretrained image prior with a predictor adapted to the current samples, allowing it to account for their distribution rather than use a single sampled noise as the reference. In Fig. 2, VSD recovers sharper burger ingredients and more vegetation around the villa than SDS. The villa's CLIP score also improves from 0.2935 to 0.3097 (Tab. 1).

**SDI.** SDI constructs the noisy latent by DDIM inversion of the current image instead of injecting fresh Gaussian noise. This makes the perturbation depend on the image and prompt, supporting more coherent updates. Its decreasing timestep schedule moves from high to low noise, while the cached target provides a fixed reconstruction objective between refreshes. In Fig. 2, SDI produces clearer cabin and church structures and cleaner surfaces than SDS. Its average CLIP score increases from 0.3042 to 0.3086.

## 5. Conclusion

We implemented SDS, VSD, and SDI for latent image optimization using Stable Diffusion 2.1-base. SDI achieved the highest average CLIP score of 0.3086, followed by SDS at 0.3042 and VSD at 0.3011. SDS produced recognizable but smoothed subjects, VSD recovered more detail with irregular textures, and SDI generally produced clearer structures and cleaner surfaces. VSD uses a learned noise predictor adapted to the current samples, while SDI obtains image-dependent perturbations through inversion, providing alternatives to the sampled-noise reference in SDS. These changes can improve visual detail, although VSD did not exceed SDS's average CLIP score.

The ablations showed that stronger SDS guidance helps only up to a point, additional inversion steps do not guarantee higher scores, and repeatedly refreshing the SDI target is preferable to fitting only its initial estimate. Directly optimizing noise-prediction MSE through the U-Net produced noisy, unrecognizable images, further illustrating the importance of the chosen update direction. Overall, the results highlight the influence of noise construction and target updates on generation quality, and the need to assess both text alignment and visual appearance.

## References


[1] Artem Lukoianov, Haitz Sáez de Ocáriz Borde, Kristjan Greenewald, Vitor Campagnolo Guizilini, Timur Bagautdinov, Vincent Sitzmann, and Justin Solomon. Score distillation via reparametrized DDIM. In *Adv. Neural Inform. Process. Syst.*, 2024. 2

[2] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *Int. Conf. Learn. Represent.*, 2023. 1, 2

[3] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *Int. Conf. Learn. Represent.*, 2021. 2

[4] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. In *Adv. Neural Inform. Process. Syst.*, 2023. 2